Code Review: `validateSystem` and nested functions (updated)

File: `SystemVM.swift`, lines 47-935 (plus `compareTotals`).

Reviewed by reading the code. The project builds (`xcodebuild`), but the validation has not been run.

Items fixed since the first review have been removed and line numbers refreshed against the current file.

Likely bugs

1. **Profit consistency checks only run when the cost totals already mismatch.** The Reference Data Student profit check (lines 256-260) is inside the `if !compareTotals(studentCost, billedStudentTotalCost)` block (line 236). The tutor version (lines 285-289) is inside the block at line 266. The check does not depend on billed data, so a bad profit figure goes unreported whenever the cost totals agree. Move both into their own loops over the Student and Tutor lists.
2. **Billed-list counts probably compare against the wrong number.** Deleted students and tutors are deliberately skipped when checking they appear in the billed list (lines 184, 208, 238, 268, 598 and 496). But line 315 compares the billed Student row count to `totalStudents`, and line 559 compares the billed Tutor row count to `totalTutors`; both include deleted records. The comments above them say "number of active". If deleted records are not in the billed sheets, any deleted record causes a false error. Confirm which is intended.
3. **`compareTotals` returns true whenever the billed total is 0 (lines 938-940).** A billed sheet that failed to load (it appears to come back as an empty result) makes every revenue and cost comparison pass. Consider treating "reference nonzero, billed zero" as a mismatch.

Consistency and cleanup

- **Duplicate checks not converted:** Tutor key and name duplicates (lines 386 and 394) still use the filter-per-item pattern (O(n^2), reports each duplicate twice). Student, service and location checks now group once.
- **Status semantics differ:** suspended tutors are exempt from the timesheet check (line 348) but counted as active (line 458) and expected to have a Details sheet. Suspended services count as deleted (line 793); suspended students count as active (line 582). Possibly intentional; add a comment.
- **Log level:** most validation errors are logged at the default level, and the Timesheet checks at lines 352-359 log at `.info`, while other failures use `.error`. Pick one convention.
- **Misleading names:** `billedStudentRevenue`, `billedStudentCost` and `billedStudentProfit` in the billed Tutor loop (lines 153-156 and 160).

- **Unused values (compiler warnings):** `tutorNumber` (381-382), `studentNumber` (639-640), `serviceNumber` (735-736), `billedTutorNum` (496), `timesheetFileID` (504), `locationNum` (606), `billedStudentNum` (323 and 598), `tutorStudentNum` (648).
- **Leftover stub:** the empty comment at line 309 ("Validate master reference spreadsheet file key...") has no code under it.

Structure

- **Repeated three-line pattern.** The log line, the `AppLogger` call and the `addMessageLine` call repeat about 60 times. One local helper, e.g. `report(_ message: String, level: LogLevel)`, would remove roughly 120 lines and make the log level consistent.
- **Hidden coupling through captured variables.** The four nested functions (starting at lines 337, 569, 709 and 826) mutate about 15 variables captured from the enclosing function, creating ordering dependencies nothing enforces. `validateLocationData` reads `activeStudents`, which only `validateStudentData` sets, and the final cross-checks read `tutorRevenue` and `locationRevenue`. A small results struct that each function fills and returns would make this explicit. Declaring the nested functions after the closing "Validation Complete" statement (line 332) also hurts readability.
- **Performance.** `validateTutorData` makes about three sequential network calls per tutor (`getFileID` at line 350, `getSheetIdByName` at 482, `fetchTutorDataCounts` at 504). The student-to-tutor assignment check (inner loop at line 647) is S x T; a dictionary of student key to assigned tutors, built once, would avoid repeated lookups. A task group would help with the network calls.